

Résilience des micro-services avec Resilience4J

Objectifs :

1. Implémenter Circuit Breaker avec Open Feign
2. Implémenter Circuit Breaker dans le Gateway

Implémenter Circuit Breaker avec Open Feign

Pour les deux micorservices department et teacher, Mettre la dépendance bus-amqp en commentaire et ses lignes dans le fichier application.yml:

```
<!-- <dependency>-->
<!-- <groupId>org.springframework.cloud</groupId>-->
<!-- <artifactId>spring-cloud-starter-bus-amqp</artifactId>-->
<!-- </dependency>-->
```

```
# rabbitmq:
# host: "localhost"
# port: 5672
# username: "guest"
# password: "guest"
```

Pour les deux micorservices department et teacher, ajouter au fichier application.yml les lignes suivantes (sous eureka au même niveau que client) :

```
eureka:
  client:
    service-url:
      defaultZone: ${DISCOVERY_SERVICE_URL:http://localhost:8761/eureka}
  instance:
    prefer-ip-address: true
    hostname: localhost
```

- Au niveau de la classe DepartmentController du projet department, Mettre en commentaire les lignes suivantes :

```
/* @Value("${build.version}")
private String buildVersion;
```

```
@Autowired
Configuration configuration;*/
```

```
/*@GetMapping("/version")
public ResponseEntity<String> version()
{
    return ResponseEntity.status(HttpStatus.OK).body(buildVersion);
}

@GetMapping("/author")
public ResponseEntity<String> retrieveAuthorInfo() {
    return ResponseEntity.status(HttpStatus.OK)
        .body(configuration.getName()+" "+configuration.getEmail() );
}*/
```

- Ajouter au pom.xml du projet teacher la dépendance suivante :

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

- Ajouter au fichier application.yml du projet teacher, sous la rubrique **spring** :

```
cloud:
  openfeign:
    circuitbreaker:
      enabled: true
```

- Ajouter au fichier application.yml du projet teacher, sur la première colonne :

```
resilience4j.circuitbreaker:
  configs:
    default:
      slidingWindowSize: 10
      permittedNumberOfCallsInHalfOpenState: 2
      failureRateThreshold: 50
      waitDurationInOpenState: 10000
```

Créer la classe DeptFallback

```
package com.nadhem.teachers.service;

import com.nadhem.teachers.entities.DepartmentDto;
import org.springframework.stereotype.Component;

@Component
public class DeptFallBack implements APIClient {
    @Override
    public DepartmentDto getDepByCode(String departmentCode) {
        return null;
    }
}
```

Modifier l'interface APIClient

```
@FeignClient(value = "DEPARTMENT", fallback = DeptFallBack.class)
public interface APIClient {
```

Modifier la méthode getTeacherById, pour éviter l'erreur NullPointerException

```
DepartmentDto departmentDto = apiClient.getDepByCode(teacher.getDepCode());
String dname;
if (departmentDto == null)
    dname="NOT AVAILABLE";
else
    dname = departmentDto.getDepName();

TeacherDto teacherDto = new TeacherDto(
    teacher.getId(),
    teacher.getFirstName(),
    teacher.getLastName(),
    teacher.getDepCode(),
    dname
);
```

<http://localhost:8081/actuator/circuitbreakers>

←

→

↺

ⓘ localhost:8081/actuator/circuitbreakers

▼ {

"circuitBreakers": {}

}

GET

▼

localhost:8081/api/teachers/1

Params

Authorization

Headers (7)

Body

Pre-req

Query Params

	Key
--	-----

Body

Cookies

Headers (5)

Test Results

Pretty

Raw

Preview

Visualize

JSON ▼

1

2

3

4

5

6

7

8

9

10

11

12

13

14

```
"teacherDto": {
  "id": 1,
  "firstName": "Nadhem",
  "lastName": "Bel Hadj",
  "depCode": "IT",
  "depName": "Info Tech"
},
"departmentDto": {
  "id": 1,
  "depCode": "IT",
  "depName": "Info Tech"
}
```

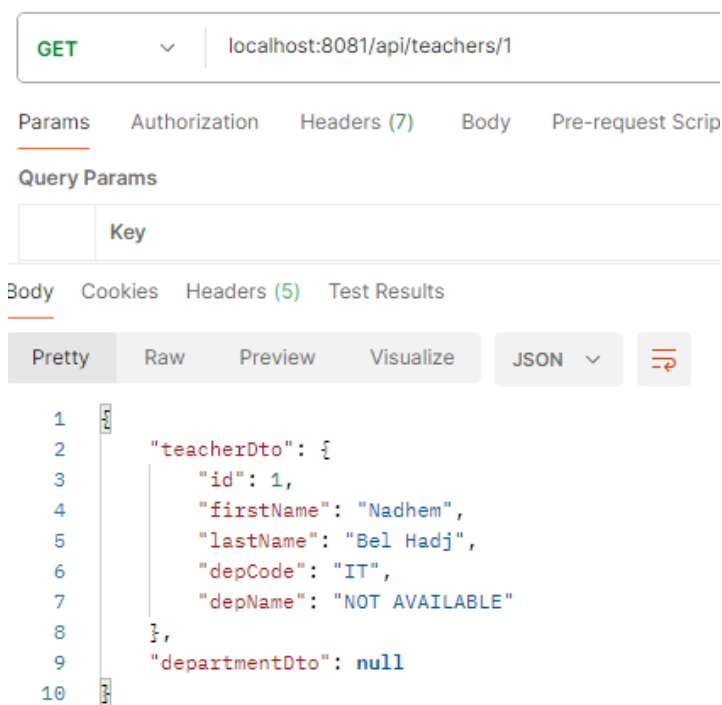
nadhemb@yahoo.com

Page 4

Spring Cloud (2025)



Arrêtez le microservice departement



Lancez 10 fois la même api (pour atteindre le threshold *failureRateThreshold* 50%)



Implémenter Circuit Breaker dans le Gateway

- Ajouter au pom.xml du projet gateway la dépendance suivante :

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-reactor-resilience4j</artifactId>
</dependency>
```

- Ajouter au fichier application.yml dans la première colonne :

```
resilience4j.circuitbreaker:
  configs:
    default:
      slidingWindowSize: 10
      permittedNumberOfCallsInHalfOpenState: 2
      failureRateThreshold: 50
      waitDurationInOpenState: 10000
```

- Commenter les routes au niveau du fichier application.yml, et décommenter la méthode *MyRouteConfig*.
- Ajouter le circuitBreaker, en modifiant la méthode *MyRouteConfig* comme suit :

```
.route(p -> p
    .path("/api/teachers/**")
    .filters( f -> f.circuitBreaker(config ->
        config.setName("teacherCircuitBreaker"))))
```

localhost:8888/api/teachers/1

<http://localhost:8888/actuator/circuitbreakers>

<http://localhost:8888/actuator/circuitbreakerevents?name=teacherCircuitBreaker>

Créer un breakpoint au niveau de la classe DepartmentServiceImpl du projet department

```
14
15 private DepartmentRepository departmentRepository; departmentRepository: "org.
16
17 @Override 1 usage
18 public DepartmentDto getDepartmentByCode(String code) { code: "IT"
19     Department dep = departmentRepository.findByDepCode(code); code: "IT" d
20
21     DepartmentDto departmentDto = new DepartmentDto(
22         dep.getId(),
23         dep.getName(),
```

localhost:8888/api/teachers/1

GET

localhost:8888/api/teachers/1

Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Query Params

	Key
	Key

Body

Cookies

Headers (2)

Test Results

Pretty

Raw

Preview

Visualize

JSON

```

1  {
2    "timestamp": "2025-04-15T15:09:10.590+00:00",
3    "path": "/api/teachers/1",
4    "status": 504,
5    "error": "Gateway Timeout",
6    "requestId": "cfd7f3af-49",
7    "message": "Did not observe any item or terminal signal within 1000ms in 'circuitBreaker'",
8    "trace": "java.util.concurrent.TimeoutException: Did not observe any item or terminal
publisher.FluxTimeout$TimeoutMainSubscriber.handleTimeout(FluxTimeout.java:296)\r\n
reactor.core.publisher.FluxTimeout$TimeoutSubscriber.onNext(FluxTimeout.java:162)\r\n\tat reactor.core.publisher.MonoDelay$MonoDelayRunnable.propagateDelay\r\n\tat reactor.core.scheduler.SchedulerTask.call(SchedulerTask.java:68)\r\n\tat reactor.core.scheduler.SchedulerTask.call(SchedulerTask.java:68)\r\n\tat java.base/java.util.concurrent.ScheduledThreadPoolExecutor.runWorker(ScheduledThreadPoolExecutor.java:1144)\r\n\tat java.base/java.lang.Thread.run(Thread.java:1583)\r\n"
9  }

```


<http://localhost:8888/actuator/circuitbreakerevents?name=teacherCircuitBreaker>

```
{
  "circuitBreakerName": "teacherCircuitBreaker",
  "type": "ERROR",
  "creationTime": "2025-04-15T16:09:10.589900100+01:00[GMT+01:00]",
  "errorMessage": "java.util.concurrent.TimeoutException: Did not observe any item or terminal signal within 1000ms in 'circuitBreaker' (and no fallback has been configured)",
  "durationInMs": 1000,
  "stateTransition": null
}
```

<http://localhost:8888/actuator/circuitbreakerevents?name=teacherCircuitBreaker>

← → ↻ ⓘ localhost:8888/actuator/circuitbreakers

```
{
  "circuitBreakers": {
    "teacherCircuitBreaker": {
      "failureRate": "10.0%",
      "slowCallRate": "0.0%",
      "failureRateThreshold": "50.0%",
      "slowCallRateThreshold": "100.0%",
      "bufferedCalls": 10,
      "failedCalls": 1,
      "slowCalls": 0,
      "slowFailedCalls": 0,
      "notPermittedCalls": 0,
      "state": "CLOSED"
    }
  }
}
```

<http://localhost:8888/actuator/circuitbreakerevents?name=teacherCircuitBreaker>

```
{
  "circuitBreakerName": "teacherCircuitBreaker",
  "type": "STATE_TRANSITION",
  "creationTime": "2025-04-15T16:21:16.337399600+01:00[GMT+01:00]",
  "errorMessage": null,
  "durationInMs": null,
  "stateTransition": "CLOSED_TO_OPEN"
},
```



Cr  er la classe FallbackController

```
package com.nadhemb.gatewayserver.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class FallBackController {

    @RequestMapping("/contactAdmin")
    public ResponseEntity<String> contactAdmin() {
        return new ResponseEntity<>("An error occurred. Please contact the administrator. !",
            HttpStatus.SERVICE_UNAVAILABLE);
    }
}
```

Modifiez la m  thode MyRouteConfig :

```
.route(p -> p
    .path("/api/teachers/**")
    .filters(f -> f.circuitBreaker(config ->
        config.setName("teacherCircuitBreaker")
        .setFallbackUri("forward:/contactAdmin")))
    )
```

GET

▼

localhost:8888/api/teachers/1

ParamsAuthorizationHeaders (7)BodyPre-request ScriptTestsSetti

Query Params

	Key
	Key

BodyCookiesHeaders (2)Test Results

PrettyRawPreviewVisualizeText ▼

1 An error occurred. Please contact the administrator. !